第 3 章　栈和队列

3.1 引言

对线性表，我们可在任意位置进行插入和删除操作，但在实际应用中，很多情况下插入和删除操作是有规则限制的。例如我们在食堂排队取餐时，显然不希望有人随意插队。再如我们乘坐电梯时，最后进入电梯靠近门口的人将最先下电梯。看来，我们可以在线性表的基础上加上一些限制规则，以更好地满足应用需求。因此栈和队列这两种数据结构就出现了。

在栈结构中，数据元素的插入和删除操作仅能在线性表的一端进行，数据元素“后进先出”；在队列中，数据元素的插入在线性表的一端，删除则在另外一端，数据元素“先进先出”。

3.2 栈的抽象数据类型

栈（stack）是仅能在表的一端进行插入和删除的线性表。允许插入和删除的一端称作**栈顶**（top），另外一端则称作**栈底**（bottom）。当栈中没有任何元素时则为空栈。

假设栈$S=(a_1,a_2,\dots,a_n)$，如图 3.1 所示，则称a_1为栈底元素，a_n为栈顶元素。栈中元素按a_1，a_2，…，a_n的顺序进栈，而退栈的顺序相反：a_n最先退出，然后a_{n-1}退出，a_1最后退出。换句话说，后进者先出。因此，栈又称为**后进先出**（Last In First Out, LIFO）的线性表。

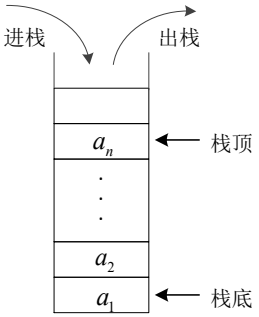


图 3.1 栈的示意图

栈的抽象数据类型定义如下。

```
ADT Stack
{
    数据对象：D = {a_i | a_i ∈ ElemSet, i = 1,2,…,n, n ≥ 0}
    数据关系：R = {< a_{i-1}, a_i > | a_{i-1}, a_i ∈ D, i = 2,…,n}约定a_n端为栈顶，a_1端为栈底。
    基本操作：
        //1. 初始化、销毁和清空操作
        InitStack( &S )
            操作结果：构造一个空栈S
        DestroyStack( &S )
            初始条件：栈S已存在
            操作结果：销毁栈S
        ClearStack( &S )
            初始条件：栈S已存在
            操作结果：将栈S重置为空栈
        //2. 访问型操作
```

StackEmpty(S)
初始条件：栈S已存在
操作结果：若栈S为空栈，则返回 TRUE，否则返回 FALSE
StackLength(S)
初始条件：栈S已存在
操作结果：返回栈S中元素个数
GetTop(S, &e)
初始条件：栈S已存在且非空
操作结果：用参数e返回栈S的栈顶元素
StackTraverse(S)
初始条件：栈S已存在且非空
操作结果：从栈底到栈顶依次逐个访问并输出栈S的数据元素
//3. 加工型操作
Push(&S, e)
初始条件：栈S已存在
操作结果：插入数据元素e为新的栈顶元素
Pop(&S, &e)
初始条件：栈S已存在且非空
操作结果：删除栈S的栈顶元素，并用参数e返回其值
} // ADT Stack

在栈的抽象数据类型定义中，与线性表不同的地方在于插入和删除操作被改成栈顶位置的压栈 Push 和弹栈 Pop 操作。

通过压栈和弹栈的操作组合，可以形成原始序列的多种输出形式。

例 3.1 给出输入序列 ABC 经过压栈和弹栈的可能输出结果。可能的结果包括：

表 3.1 ABC 出栈可能的序列

出栈后的序列	操作步骤
CBA	Push(S,‘A’),Push(S,‘B’),Push(S,‘C’),Pop(S,e),Pop(S,e),Pop(S,e)
ABC	Push(S,‘A’),Pop(S,e),Push(S,‘B’),Pop(S,e),Push(S,‘C’),Pop(S,e)
ACB	Push(S,‘A’),Pop(S,e),Push(S,‘B’),Push(S,‘C’),Pop(S,e),Pop(S,e)
BAC	Push(S,‘A’),Push(S,‘B’),Pop(S,e),Pop(S,e),Push(S,‘C’),Pop(S,e)
BCA	Push(S,‘A’),Push(S,‘B’),Pop(S,e),Push(S,‘C’),Pop(S,e),Pop(S,e)

但出栈序列 CAB 是不可能发生的，因为第一个输出是 C 说明 A、B 均已压栈且 A 在栈底，所以 A 不可能先于 B 被弹出。

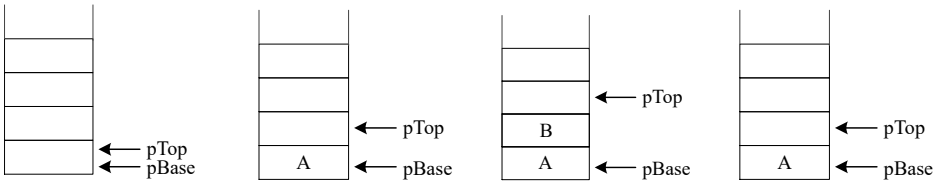
3.3 栈的顺序表示与实现

顺序栈（sequential stack）是栈的顺序存储表示，可用一块连续的存储单元作为其存储结构。与线性表类似，该存储空间既可以是静态分配的固定大小的一维数组，也可以是动态分配的一块连续存储空间。由于静态分配存在的栈满溢出的可能，所以这里就只讨论动态分配连续空间的方式。

顺序栈的动态存储描述方式如下：

```
#define STACKINITSIZE      256    // 初次分配空间大小
#define STACKINCREMENT   128    // 空间分配增量大小
typedef struct SeqStack
{
   ElemType *pBase;                  // 动态存储空间的基地址，作为栈底
   ElemType *pTop;                   // 栈顶指针，指向真实栈顶元素的下一个位置
   int   stacksize;                  // 当前已分配的存储空间大小
}SeqStack;
```

在这里，pBase 既是动态分配空间的基地址也是栈底指针，始终指向栈底位置。pTop 为栈顶指针，这里为了操作的方便，令其指向真实栈顶元素的下一个位置。当 pTop==pBase 时为空栈，pTop-pBase 值则为栈中元素的个数。具体如图 3.2 所示。



a) 空栈　　b）‘A’压栈　　c）‘B’压栈　　d）‘B’弹栈

图 3.2 顺序栈示意图

根据栈抽象数据类型中给出的基本操作，采用顺序栈的动态存储方式，给出具体实现的伪代码。

1. 初始化、销毁和清空操作

算法 3.1 顺序栈初始化

```
Status InitStack( SeqStack &S )
{  // 初始化顺序栈
   S.pBase = (ElemType *)malloc(STACKINITSIZE*sizeof(ElemType));   // 申请存储空间
   if( S.pBase == NULL )  exit(OVERFLOW);                          // 存储空间申请失败
   S.pTop = S.pBase;                                               // 栈顶指针同时指向栈底
   S.stacksize = STACKINITSIZE;                                    // 当前已分配的存储空间大小
   return OK;
} // InitStack
```

算法 3.2 顺序栈销毁

```
Status DestroyStack( SeqStack &S )
{    // 销毁顺序栈
     if( S.pBase != NULL )
     {
          free(S.pBase);
          S.pBase = NULL;
     }
     S.pTop       = NULL;
     S.stacksize  = 0;
     return OK;
}  // DestroyStack
```

算法 3.3 顺序栈清空

```
Status ClearStack( SeqStack &S )
{    // 清空顺序栈
     S.pTop  = S.pBase;
     return OK;
} // ClearStack
```

从算法 3.2 和算法 3.3 的实现代码可以看出顺序栈销毁和清空操作的区别。在进行清空操作时只是将 S.pTop 指向 S.pBase 表示清空栈中数据元素，但其占用的存储空间并未释放。

2. 访问型操作

顺序栈访问型操作的实现较为简单。下面仅给出访问栈顶元素的操作，其他操作可由读者自行实现。

算法 3.4 访问顺序栈栈顶元素

```
Status GetTop( SeqStack S, ElemType &e )
{  // 若栈不空，用参数e返回栈S的栈顶元素；否则返回 ERROR
   if ( S.pTop==S.pBase ) return ERROR;  // 空栈，无数据元素可访问
   e  = *(S.pTop-1);                     // 将真实栈顶元素赋值给 e
   return OK;
} // GetTop
```

在算法 3.4 中，我们只是将栈顶数据元素赋值给 e，并观察栈顶元素值但没有将栈顶元素从栈中弹出，注意它与 Pop 操作的区别。

3. 加工型操作

顺序栈加工型操作主要包括压栈 Push 和弹栈 Pop，见算法 3.5 和算法 3.6。

算法 3.5 顺序栈压栈操作

```
Status Push( SeqStack &S, ElemType e )
{    // 插入数据元素 e 作为新的栈顶元素
     if(  S.pTop-S.pBase >= S.stacksize )        // 当前存储空间已满，需增加存储空间
     {
          S.pBase = ( ElemType* ) realloc( S.pBase, (S.stacksize+STACKINCREMENT)*sizeof(ElemType) );
          if ( S.pBase == NULL )  exit(OVERFLOW); // 内存申请失败
          S.pTop = S.pBase + S.stacksize;               //  重新计算栈顶指针位置
          S.stacksize  +=  STACKINCREMENT;
     }
     *S.pTop  =  e;     // 插入元素 e 作为新的栈顶元素
     S.pTop++;          // 栈顶指针指向真实栈顶的下一个位置
     return OK;
} // Push
```

算法 3.6 顺序栈弹栈操作

```
Status Pop( SeqStack &S, ElemType &e )
{    // 若栈不空，则删除 S 的栈顶元素并用 e 返回，然后返回 OK；否则返回 ERROR
     if( S.pTop == S.pBase )  return ERROR;  // 空栈，无数据可弹出
     e = *(S.pTop-1);                        // 将栈顶元素赋值给 e
     S.pTop--;                               // 栈顶指针指向新的栈顶位置
     return OK;
} // Pop
```

对于顺序栈来说，在栈顶位置（相当于线性表表尾）进行压栈和弹栈不需要移动其他元素，因此该操作的时间复杂度为$O(1)$。这恰好是顺序表插入和删除时的最好情况。

3.4 栈的链式表示与实现

采用链式表示的栈称为**链式栈**（linked stack），如图 3.3 所示。显然，为了方便压栈和弹栈的操作，应将栈顶设在单链表的表头，并将栈底放在单链表的表尾。这样，压栈和弹栈的时间复杂度也为$O(1)$。

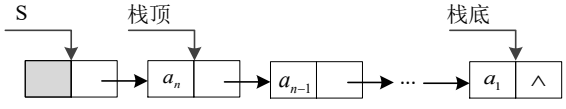


图 3.3　链式栈结构示意图

链式栈结点的存储结构与单链表结点一致，定义如下：

```
typedef struct LNode
{
   ElemType       data;          // 数据域
   struct LNode   *next;         // 指针域
} LNode, *LinkStack;
```

链式栈的大部分操作与单链表操作类似，这里就不再一一列出。仅给出压栈和弹栈的操作方法。

算法 3.7 链式栈的压栈操作

```
Status Push( LinkStack &S, ElemType e )
{    // 将数据元素 e 压栈
     //1. 申请新的结点 s
     s = (LNode *) malloc( sizeof(LNode) );
     if ( s == NULL ) exit(OVERFLOW); // 内存申请失败
     s->data   = e;
     //2. 将结点 s 链接到链式栈的头结点之后
     s->next   = S->next;
     S->next  = s;
     return OK;
} //Push
```

算法 3.8 链式栈的弹栈操作

```
Status Pop( LinkStack &S, ElemType &e )
{    // 将栈顶数据元素弹出，并用 e 返回
     if ( S->next == NULL ) return ERROR;  // 空栈，无数据元素可弹出
     //1. 将栈顶数据元素赋值给 e
      e = S->next.data;
     //2. 删除栈顶元素
     p = S->next;
     S->next = p->next;
     free(p);
     return OK;
} //Pop
```

可见，针对压栈和弹栈操作，可充分利用顺序表和链表的特性，将栈顶置于特定的位置，以避免数据的移动操作。

在一些特定场合下，可以利用一个连续存储空间同时实现两个栈。这两个栈的栈底分别设置在数组的两端，栈顶则向数组中间移动，因此称作**双端栈**。这样，两个栈可以共享同一个存储空间，互相调剂且灵活性强。

3.5 栈的应用举例

栈的最大特性就是“后进先出”，栈的应用与此密切相关。例如，与数制转换相关的逆序输出，与括号检验、表达式求值相关的最近匹配与比较，与迷宫问题、八皇后问题相关的递归与回溯等。本小节的应用举例将围绕这些典型问题展开。

3.5.1 逆序输出问题

生活中有许多逆序问题的实例。例如，冬天我们常穿多件衣服时，那么脱衣的顺序刚好是穿衣的逆序；电梯在下行时经过的楼层是上行时经过楼层的逆序；中国古诗文中也有“天连水尾水连天”这样正反读都一样的回文。在计算机应用中，十进制与八进制、十六进制转换也是一个典型的逆序输出问题。这里以回文判断进行举例。

例 3.2 回文判断 正读和反读都相同的字符序列称为回文。例如，“ abcba ”是回文，“abcde”则不是回文。试给出一个算法，判别给定的字符串是否为回文。

从回文的定义可以看出，回文字符串的正序和逆序输出结果是相同的。算法需要做的就是将输入字符串进行栈逆序操作，然后将其与正序字符串进行比较，判断两者是否相同。

算法 3.9 回文字符串判断算法

```
Status CheckPalindrome( char *str )
{   // 检查 str 字符串是否是回文，是回文返回 RET_YES；否则返回 RET_NO
    InitStack( S );
    //1. 将字符串逐个字符压栈
    str_len = strlen(str);
    for( i=0; i<str_len; i++ )
    {
        Push(S, str[i]);
    }
    //2. 从栈中逐个弹出字符并将其与字符串的正序序列进行比较
    tag = RET_YES;              // 是否为回文标记，初始化为 RET_YES
    for( i=0; i<str_len; i++ )
    {
        Pop(S, e);
        if ( e!=str[i] )
        {
            tag = RET_NO;    // 不是回文
            break;
        }
    }
    Destroy(S);
    return tag;
} //CheckPalindrome
```

3.5.2 最近匹配与比较问题

我们在写 C/C++代码时，经常会出现括号疏漏、不匹配等问题。这时编译器程序会及时报告错误并提示。那么编译器是如何做到的呢？

例 3.3 括号匹配检验 以算法 3.9 代码为例，一段代码会涉及圆括号（parentheses, “( )”）、方括号（square brackets, “[ ]”）和花括号（curly braces, “{ }”）。这些括号成对出现，而且符合最近匹配规则。例如，在“ { ([ ]) } ”中，每次出现的右括号就是跟最近的左括号进行配对。如果出现漏写、错写则会导致程序编译不通过。

显然，要进行最近匹配，栈是合适的结构。每当遇到一个括号字符时，如果是左括号，则进栈；如果是右括号，则将其与栈顶的左括号进行匹配，若匹配成功则将该左括号弹栈。正确的情况是，遍历完程序代码字符串后所有括号都能匹配且栈空。出错的情况则会有好几种，需要分情况讨论：

（1）当前的右括号与栈顶的左括号不属于同一类型。例如，字符串“ ([ )] ”的栈顶是‘[’，但与其进行最近匹配的却是‘) ’。

（2）直到代码字符遍历结束也没有遇到待匹配的右括号。例如，“{ ( )”缺少右括号是常见的代码问题。

（3）栈已空却还有多余的右括号。例如，“{ ( ) } } ”多出了一个不该出现的右括号。

至此，我们已列举了正确和错误的情况。接下来考虑将其组合成完整的算法描述。显然，算法执行是依照代码字符串遍历的顺序进行的，算法的具体描述如下。

依次读取代码字符串中的字符：

若是非括号字符，则忽略并继续读取下一字符；

若是左括号，则压栈；

否则为右括号，与栈顶元素进行匹配：

若栈空，则是多余的右括号（出错情况 3），报错返回；

否则

若匹配成功，则栈顶元素弹栈，继续读取下一个字符；

否则匹配错误（出错情况 1），报错返回。

代码字符串遍历结束时：

若栈空，则匹配成功，正确返回；

否则有左括号还没匹配（出错情况 2），报错返回。

上述过程描述了按照字符串遍历顺序对左右括号进行分情况处理以及通过压栈和弹栈操作进行匹配检验，涵盖了出错的情况和成功匹配的情况。该描述可较为方便地转换成算法伪代码表示。该思想也可以扩展到 HTML 文档中标志对的匹配检验。

算法 3.10 括号匹配检验算法

```
Status CheckParenMatching( char *str  )
{   // 检查 str 字符串中括号是否匹配，完全正确返回 RET_OK；否则返回不同形式的 ERROR
    InitStack( S );
    // 逐字符检查是否是括号
    str_len = strlen(str);
    tag = RET_OK;
    for( i=0; i<str_len && tag==RET_OK; i++ )
    {
        switch ( str[i] )
        {
            case '(': case  '[': case '{':       // 若是左括号，则压栈
                      Push(S,str[i]);  break;
            case ')':
                      if ( StackEmpty(S) )    // 栈已空，有多余的右括号
                      { tag = RET_ERROR_EXTRA;  break; }
                      Pop(S, e);
                      if ( e != '(' ) tag =  RET_ERROR_NOTMATCH;    // 不匹配
                      break;
            case ']':
                      if ( StackEmpty(S) ) // 栈已空，有多余的右括号
                      {  tag =  RET_ERROR_EXTRA; break; }
                      Pop(S, e);
                      if ( e != '[' ) tag =  RET_ERROR_NOTMATCH;    // 不匹配
                      break;
            case '}':
                      if ( StackEmpty(S) ) // 栈已空，有多余的右括号
                      {  tag =  RET_ERROR_EXTRA; break; }
                      Pop(S, e);
                      if ( e != '{' ) tag = RET_ERROR_NOTMATCH;     // 不匹配
                      break;
             default:  break;                    // 其他字符，则继续下一个
        } // end switch
    }  // end for

    if ( ! CheckEmpty(S) && tag == RET_OK )  tag =  RET_ERROR_EXTRA;   //栈已空，有多余的左括号
    DestroyStack(S);
    return tag;
} //CheckPalindrome
```

例 3.4 算术表达式求值 除了括号的匹配检验，还有一个典型的例子就是算术表达式求值。例如“3+4×2 - (1+1)# ”这样的表达式计算。在计算过程中，不是读到一个运算符就立即计算，而是

要与后面的运算符进行优先级比较，以决定先算哪个。换言之，当前读到的运算符要与前面最近的运算符进行优先级比较。栈结构非常合适用来进行这样的最近比较。

首先要进行一些约定。表达式是由操作数（operand）、运算符（operator）和界限符（delimiter）组成。操作数和运算符是表达式的主要组成部分，界限符标志了一个表达式的结束。表达式可分为 3 类，即算术表达式、关系表达式和逻辑表达式。本例只讨论算术表达式的计算，并对表达式做如下简化：

（1）假定所有运算数都为整数。

（2）所有运算符都是整数的二元操作，且都用一个字符表示。

其次，确定加减乘除四则运算的规则：先乘除后加减；从左向右计算；先算括号内，再算括号外。基于此，相邻的前、后运算符之间的优先级关系如表 3.2 所示，其中θ_1代表前面的运算符，θ_2代表当前的运算符。例如，θ_1为 + ，θ_2 为 - 时，对应的优先级关系为“>”，则表示θ_1的运算符优先级更高，要先进行计算。优先级表中为空白的项表示该种情况不应出现，为了突出算法的主要思想，假定输入的表达式合法无错误，即不会出现这样的情况。

表 3.2　相邻的前、后运算符之间的优先级关系

θ_1 \ θ_2	+	—	×	/	(	)	#
+	>	>	<	<	<	>	>
—	>	>	<	<	<	>	>
×	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=

再次，根据上述约定，利用操作数栈（OPND 栈）和运算符栈（OPTR 栈），设计对应的计算方法：

（1）初始化：置 OPND 栈为空栈，OPTR 栈的栈底元素为表达式起始符’#’；

（2）执行过程：

依次读入表达式中每个字符：

若为操作数，则转换为对应数值压入 OPND 栈；

否则为运算符，和 OPTR 栈的栈顶运算符进行优先级比较：

若栈顶运算符优先级低于当前运算符，则当前运算符压栈；

若栈顶运算符优先级高于当前运算符，则 OPTR 栈弹出运算符、OPND 栈弹出两个运算数，进行相应计算，并将计算结果压入 OPND 栈；

若栈顶运算符优先级等于当前运算符，则当前运算符为右括号，将对应的左括号从 OPTR 中弹栈。

执行结束，将 OPND 栈栈顶元素作为运算结果返回。

最后，根据上述计算方法可以较为方便地转换为对应的算法 3.11。

算法 3.11　表达式求值算法

```
Status CalculateExpression( char *str , double &result)
{   // 利用 OPND 和 OPTR 栈进行算法表达式求值。

    InitStack(OPND);
    InitStack(OPTR);  Push(OPTR, '#');

    // 依次读入表达式每个字符
```

```
    i = 0;
    GetTop(OPTR, theta1);
    while( str[i] != '#' || theta1!='#' )
    {
        if ( isDigit(str[i]) ) // 若为操作数，则转换为对应数值压入 OPND 栈
        {
            readNumber( &str[i] , data);  // 将字符型表示转换为对应数值，并将 i 顺次移动
            Push(OPND, data);             // 压入 OPND 栈
        }
        else                  // 否则为运算符
        {
            GetTop(OPTR, theta1);         // 查看 OPTR 栈顶运算符 theta1
            // 按照表 3.2，进行栈顶元素 theta1 和 当前运算符 theta2 优先级比较
            switch ( ComparePriority( theta1, str[i] ) )
            {
                case '<':   // 栈顶运算符优先级低，则当前运算符压栈
                        Push(OPTR,str[i]); i++; break;
                case '>':   // 栈顶运算符优先级高，则进行计算，计算结果压入 OPND 栈
                        POP(OPTR, theta); Pop(OPND, b); Pop(OPND, a);
                        Push(OPND, Operate(a, theta, b));
                        break;
                case '=':   // str[i]中为右括号，脱括号
                        Pop(OPTR, theta);  i++;  break;
                 default: break;
            } // end switch
        } // end else
    }  // end while

    GetTop(OPND, result); // 将计算结果赋值给 result
    return RET_OK;
} //CalculateExpression
```

以“3+4×2 - (1+1)#”为例，在执行算法 3.11 时，OPND 和 OPTR 栈的变化情况如表 3.3 所示。

表 3.3　“3+4×2 - (1+1)#”执行过程

步骤	OPND 栈	OPTR 栈	输入字符	主要操作
1		#	3+4×2 - (1+1)#	Push(OPND, 3)
2	3	#	+4×2 - (1+1)#	Push(OPTR,'+')
3	3	# +	4×2 - (1+1)#	Push(OPND, 4)
4	3 4	# +	×2 - (1+1)#	Push(OPTR,'×')
5	3 4	# + ×	2 - (1+1)#	Push(OPND, 2)
6	3 4 2	# + ×	- (1+1)#	Operate(4,'×',2)
7	3 8	# +	- (1+1)#	Operate(3,'+',8)
8	11	#	- (1+1)#	Push(OPTR,'-')
9	11	# -	(1+1)#	Push(OPTR,'(')
10	11	# - (	1+1)#	Push(OPND,1)
11	11 1	# - (	+1)#	Push(OPTR,'+')
12	11 1	# - (+	1)#	Push(OPND,1)
13	11 1 1	# - (+	)#	Operate(1,'+',1)
14	11 2	# - (	)#	Pop(OPTR,theta)
15	11 2	# -	#	Operate(11,'-',2)
16	9	#	#	return(GetTop(OPND))

在计算机的编译器中，针对表达式求值问题还有一种解决方法，即将正常的中缀表达式转换为后缀表达式，然后对后缀表达式进行求解。同样以“3+4×2 - (1+1)”为例，其后缀表达式为“3 4

2 × + 1 1 + -”。针对这种表达式，在计算时每读到一个运算符，就连同前面的两个运算数进行计算，该式执行过程如表 3.4 所示。

表 3.4 “3 4 2 × + 1 1 + - #”后缀表达式执行过程

步骤	OPND 栈	输入字符	主要操作
1		3 4 2 × + 1 1 + - #	Push(OPND, 3)
2	3	4 2 × + 1 1 + - #	Push(OPND, 4)
3	3 4	2 × + 1 1 + - #	Push(OPND, 2)
4	3 4 2	× + 1 1 + - #	Operate(4,‘×’,2)
5	3 8	+ 1 1 + - #	Operate(3,‘+’,8)
6	11	1 1 + - #	Push(OPND, 1)
7	11 1	1 + - #	Push(OPND, 1)
8	11 1 1	+ - #	Operate(1,‘+’,1)
9	11 2	- #	Operate(11,‘-’,2)
10	9	#	return(GetTop(OPND))

可见，后缀表达式的计算中，由于省去了操作符优先级的比较，所以计算过程相对简单。如何从中缀表达式生成后缀表达式，读者可参考相关教材，这里不再赘述。

3.5.3 递归与回溯问题

计算机程序运行时，其函数调用的顺序也遵循“后进先出”的原则，最后被调用的函数将最先返回。计算机系统使用系统栈来实现函数调用的现场保存与恢复。

特别地，递归（recursion）是函数直接或间接调用自身的一种形式。递归可通过将一个大问题层层转化为一个与原问题相似的规模较小的问题来求解，只需较少的代码即可解决复杂的问题。当然，递归需要有边界条件的约束，从而能够正确返回。同时，受系统栈的容量限制，递归的深度不能太深。显然，可以使用自定义的数据栈，并将递归函数改成非递归函数，以避免递归深度的限制。

回溯是算法设计的基本思想之一，即从一条路往前走，能进则进，不能进则退回来，换一条路再试。可见，“退回来”符合栈的特性，也符合递归的特性。

本节将以 N 皇后问题为例，讨论回溯问题的递归与非递归解决方法。

例 3.5 N 皇后问题 我们以 4 皇后问题为例给出问题的定义。4 皇后问题是指在 4×4 的国际象棋棋盘中，放置 4 个“皇后”棋子，同时满足这样的条件：任意两个皇后不出现在同一行、同一列、同一正斜线和同一逆斜线上。如图 3.4 所示，在与皇后棋子相关联的阴影格中不可再放其他皇后。

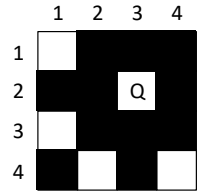


图 3.4 皇后势力范围示意图

本例采用回溯法的思想进行 4 皇后放置（过程见图 3.5）。假设第 1 个皇后放在第 1 行第 1 列(图 3.5(1))，则第 2 个皇后可放置的第一个候选位置为第 2 行第 3 列(图 3.5(2))。这样，第 3 个皇后在第 3 行就没有候选位置(图 3.5(3))，需“回退”到第 2 行重新摆放第 2 个皇后，改为第 2 行第 4 列(图 3.5(4))，则第 3 个皇后则可放在第 3 行第 2 列(图 3.5(5))。这时，第 4 个皇后在第 4 行也无法放置(图 3.5(6))，需“回退”并重新摆放第 3 个皇后，同样也无可选位置(图 3.5(7))。继续回退并重新摆放第 2 个皇后，无可选位置(图 3.5(8))，因此继续回退到第 1 行，将第 1 个皇后放到第 1 行第 2

列(图 3.5(9))。然后依次再放置其他皇后，当第 2 个皇后在第 2 行第 4 列(图 3.5(10))、第 3 个皇后在第 3 行第 1 列(图 3.5(11))、第 4 个皇后在第 4 行第 3 列(图 3.5(12))时，得到符合条件的一个解。

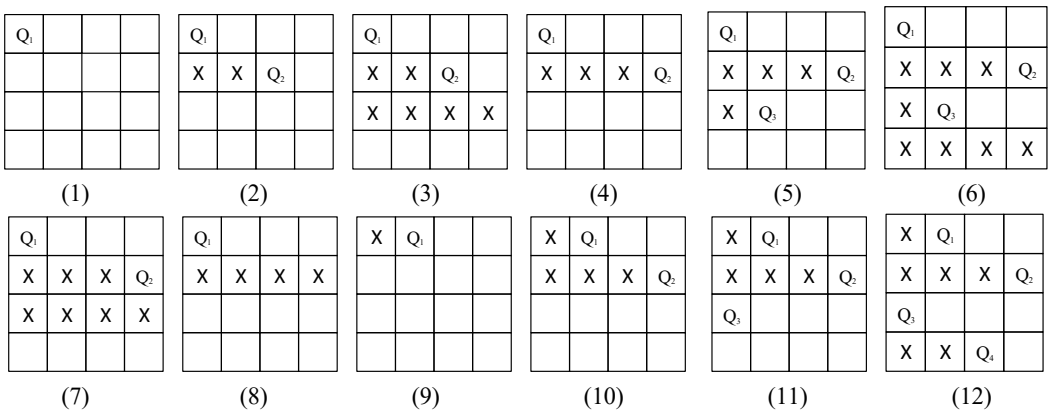


图 3.5 4 皇后放置过程示意图

根据上面的分析，首先设计用来表示皇后的数据结构，采用顺序栈的方式存储皇后的位置。

```
typedef struct
{
   int col;       // 列坐标值
   int row;       // 行坐标值
}Point;
typedef Point ElemType;  // 数据元素为皇后的位置坐标值
```

然后，考虑是否存在皇后冲突的判别方法。由于每行只放一个皇后，所以只要判断新放置的皇后跟前面已放置的皇后是否存在同一列、同一正斜线和同一逆斜线上。对于是否在同一列的判断，就是看新皇后位置的列坐标跟栈中其他皇后的列坐标是否相等。对于同一正、逆斜线上的两个点，其斜率为 1 或-1，即|y2-y1|/|x2-x1| = 1，化简为|y2-y1|=|x2-x1|。因此，相应的判断冲突的函数如算法 3.12 所示。

算法 3.12 皇后冲突判断算法

```
Status JudgeQueenConfliction( Point newQueen , SeqStack StkQueen)
{   // 判断新的皇后位置与已有皇后位置是否有冲突，没有冲突返回 RET_OK，有冲突返回 RET_CONFLICT
    ElemType  *pCurQueen;

    x1  = newQueen.col;
    y1  = newQueen.row;
    pCurQueen = StkQueen.pBase;
    tag  = RET_OK;
    while( pCurQueen < StkQueen.pTop )
    {
        x2 = pCurQueen->col;
        y2 = pCurQueen->row;
        if ( x1==x2 ) { tag = RET_CONFLICT; break; }                    // 同一列
        if ( abs(x1-x2) == (y1-y2) )  { tag = RET_CONFLICT; break; }   // 同一正斜线或逆斜线
        pCurQueen++;
    } //end while

    return tag;
} //JudgeQueenConfliction
```

最后，给出 4 皇后的算法思想。

（1）初始状态：将第 1 个皇后放置到第 1 行第 1 列，并压栈；当前处理行编号置为 2，列编号置为 1。

（2）执行过程：

当前处理的行编号不大于且列编号不大于 4 ：

从该行当前列编号开始，检查是否有冲突；

若找到不冲突的位置，则：

放置皇后，将位置压栈；当前处理行编号增加 1，列编号置为 1；

当前行编号大于 4 时：

本轮全部放置完毕，输出 4 个皇后的位置。弹栈，修改最后一个皇后的位置，进行下一种解的试探。

否则：

将栈顶皇后弹栈，当前处理行改为该皇后所在行，当前列编号置为该皇后所在列的后一列，若列发生越界，则继续执行弹栈处理直到找到合适位置。

当然，该方法可以很自然地推广到 8 皇后问题甚至 N 皇后问题。根据上述算法思想，给出 N 皇后的放置算法实现（算法 3.13）。

算法 3.13 N 皇后放置非递归算法

```
Status PlaceQueen( int N )
{   // N 皇后问题非递归实现
    SeqStack StkQueen;
    Point      curQueen;

    InitStack(StkQueen);
    resultCount  =  0;              //满足要求的解数量置为 0
    curQueen.row = 1;
    curQueen.col  = 1;
    Push(StkQueen, curQueen);  // 将第 1 个皇后放置在第 1 行第 1 列

   curQueen.row = 2;
   curQueen.col   =1;
   while (  curQueen.row <= N && curQueen.col <= N )
   {
       while ( curQueen.col <= N )
       {
            ret = JudgeQueenConfliction( curQueen , StkQueen);
            if ( ret == RET_OK)  break;       // 在本行上找到了合适的位置
            curQueen.col = curQueen.col+1;  // 存在冲突，则试探下一个位置
        } // end while  ( curQueen.col <= N )
       if ( ret == RET_OK )   // 不冲突
       {
           Push(StkQueen, curQueen);
           curQueen.row = curQueen.row +1;
           curQueen.col   = 1;
           if ( curQueen.row > N )  // 当前放置的皇后已经满足要求
           {
               OutputResult( StkQueen , N );     //根据栈中皇后的位置，输出满足要求的 N 皇后放置结果
               resultCount++;                        // 解的数目加 1
               Pop( StkQueen, curQueen);         // 弹栈
               curQueen.col = curQueen.col+1;  // 从当前的下一个位置继续试探
               while ( curQueen.col > N && !StackEmpty(StkQueen) )   // 若回退不满足要求，则继续回退
               {
```

```
                    Pop(StkQueen, curQueen);
                    curQueen.col = curQueen.col + 1;
                } // end while
            } // end if ( curQueen.row > N )
        } // end if ( ret == RET_OK )  // 不冲突
        else  // 本行上未找到合适位置，回退
        {
            Pop(StkQueen, curQueen);
            curQueen.col = curQueen.col + 1;
            while ( curQueen.col > N && !StackEmpty(StkQueen) )  // 若回退不满足要求，则继续回退
            {
                Pop(StkQueen, curQueen);
                curQueen.col = curQueen.col + 1;
            } // end while
        }  // end else
    } //end while  (  curQueen.row <= N && curQueen.col <= N )

    cout<<resultCount;      // 符合条件解的总个数
    DestroyStack( StkQueen );
    return RET_OK;
} // PlaceQueen
```

相应地，递归方法可以让代码简洁一些。

算法 3.14 N 皇后放置递归算法

```
Status PlaceQueen( int row )
{   // N 皇后问题递归实现

    for( col=0; col<N; col++)
    {
        curQueen.row = row;
        curQueen.col  = col;
        Push(StkQueen, curQueen);
        ret = JudgeQueenConfliction( curQueen , StkQueen);
        if ( ret == RET_OK)
        {
            if ( row < N-1 )  PlaceQueen( row+1 );  //未到达最后一行，继续递归下一行
            else
            {
                OutputResult( StkQueen , N );  //根据栈中皇后的位置，输出满足要求的 N 皇后放置结果
                resultCount++;
            }
        } //end if
        Pop(StkQueen, curQueen);
    }  // end for

    return RET_OK;
} // PlaceQueen
```

当然，也可以用穷举搜索的方法来寻找 N 皇后问题可能的解，读者可自行比较分析回溯法和穷举法在时间复杂度上的差异。

该类问题还包括斐波拉契数列求值、汉诺塔、迷宫行走等经典问题，读者可参阅相关文献自行学习。

3.6 队列的抽象数据类型

队列（queue）也是一种特殊的线性表，它是一种只允许在表的一端进行插入操作、在表的另一端进行删除操作的线性表。表中允许进行插入操作的一端称为**队尾**（rear），允许进行删除操作的一端称为**队头**（front / head）。队列的插入操作通常称为入队，队列删除操作通常称为出队。当队列中没有元素时，称其为**空队列**。

假设队列 $Q=(a_1,a_2,\dots,a_n)$, 如图 3.6 所示，则称a_1为队头元素，a_n为队尾元素。队列与现实生活中的等车、买票时的排队相似，新来的成员总是加在队尾，每次离开的总是队头的成员。队列中元素按a_1，a_2，…，a_n的顺序入队，也按照该顺序出队，因此队列又称为**先进先出**（First In First Out, FIFO）的线性表。

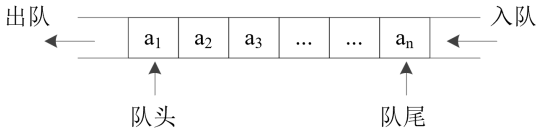


图 3.6 队列的示意图

队列的抽象数据类型定义如下。

```
ADT Queue
{
```

数据对象： D = $\{a_i | a_i \in ElemSet, i=1,2,\dots,n, n\geq 0\}$

数据关系： R = $\{<a_{i-1},a_i> | a_{i-1},a_i \in D, i=2,\dots,n\}$约定$a_1$端为队头，$a_n$端为队尾

基本操作：

//1. 初始化、销毁和清空操作

InitQueue(&Q)

操作结果：构造一个空队列 Q

DestroyQueue(&Q)

初始条件：队列 Q 已存在

操作结果：销毁队列 Q

ClearQueue(&Q)

初始条件：队列 Q 已存在

操作结果：将队列 Q 重置为空队列

//2. 访问型操作

QueueEmpty(Q)

初始条件：队列 Q 已存在

操作结果：若队列 Q 为空队列，则返回 TRUE，否则返回 FALSE

QueueLength(Q)

初始条件：队列 Q 已存在

操作结果：返回队列 Q 中元素个数

GetHead(Q, &e)

初始条件：队列 Q 已存在且非空

操作结果：用参数e返回队列 Q 的队头元素

QueueTraverse(Q)

初始条件：队列 Q 已存在且非空

操作结果：从队头到队尾依次访问并输出队列 Q 的数据元素

//3. 加工型操作

EnQueue(&Q, e)

初始条件：队列 Q 已存在

操作结果：插入数据元素e作为新的队尾元素

DeQueue(&Q, &e)

初始条件：队列 Q 已存在且非空

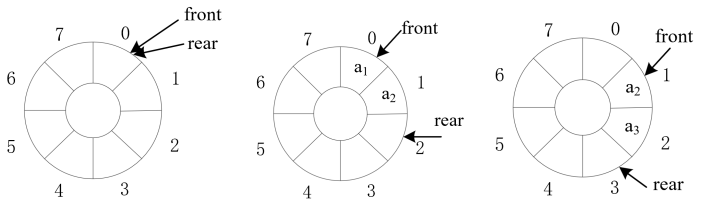
操作结果：删除队列 Q 的队头元素，并用参数e返回其值

} // ADT Queue

相对于线性表的插入和删除操作，队列中分别为队尾的入队 EnQueue 操作和队头的出队 DeQueue 操作。

3.7 队列的顺序表示与实现

由于队列的入队和出队分别在线性表的两端，不同于栈的一端操作，因此如果直接用顺序表来表示队列，则出队操作较为麻烦，每次出队都要移动整个队列的元素，但不移动又会浪费顺序表前面的存储空间。为了解决这个问题，人们提出了**循环队列**（circle queue）的思想，即将顺序表的表头和表尾连在一起，同时用 front 和 rear 两个指针分别指向循环队列的队头和队尾。这样，顺序表中的存储区域就自然被分成两部分：队列占用部分和空闲空间部分。队列的头尾刚好在空闲空间的两端，因此入队和出队操作就都不需要移动队列元素本身了，如图 3.7 所示。



a) 空队列　　b) a1, a2 入队　　c) a1 出队，a3 入队

图 3.7 循环队列示意图

循环队列的描述方式如下：

```
#define MAXQSIZE        256    // 最大队列长度
typedef struct SeqQueue
{
   ElemType *pBase;             // 动态存储空间的基地址
   int   front;                 // 队头指针，若队列不空，指向队头元素
   int   rear;                  // 队尾指针，若队列不空，指向队尾的下一个位置
}SeqQueue;
```

由定义可见：当 front==rear 时为空队列；当(rear+1)%MAXQSIZE==front 时，队尾追上了队头，则队列满；队列的长度可表示为(rear+MAXQSIZE-front)%MAXQSIZE，这种计算方法巧妙地处理了头尾循环相接的问题，就像从钟表盘面上计算下午 1 点到上午 11 点之间的小时数一样。

另外，也有教材中将 rear 指针指向真实的队尾，这样就需要增加一个 length 变量来记录队列长度。读者在阅读相关教材时注意区分。

这里简单给出基于此存储结构的循环队列初始化、入队和出队操作的实现，其他实现由读者自行完成。

1. 初始化操作

算法 3.15 循环队列的初始化操作

```
Status InitQueue( SeqQueue &Q )
{   // 初始化循环队列
    Q.pBase = (ElemType *)malloc(MAXQSIZE*sizeof(ElemType));   // 申请存储空间
    if( Q.pBase == NULL )  exit(OVERFLOW);                     // 存储空间申请失败
    Q.front = 0;               // 空队列，队头指向 0 号单元
    Q.rear  = 0;               // 空队列，队尾指向 0 号单元
    return OK;
} // InitQueue
```

2. 加工型操作：入队和出队

循环队列加工型操作主要包括入队 EnQueue 和出队 DeQueue，见算法 3.16 和算法 3.17。

算法 3.16 循环队列的入队操作

```
Status EnQueue( SeqQueue &Q, ElemType e )
{    // 插入数据元素 e 为新的队尾元素
     if( (Q.rear+1)%MAXQSIZE == Q.front )        // 队列满
          return ERROR;
     Q.pBase[Q.rear] =  e;                  // 插入元素 e 作为新的队尾元素
     Q.rear = (Q.rear+1)%MAXQSIZE;      // 队尾指针指向队尾元素的下一个位置
     return OK;
} //EnQueue
```

算法 3.17 循环队列的出队操作

```
Status DeQueue( SeqQueue &Q, ElemType &e )
{    // 若队列不空，则删除 Q 的队头元素并用 e 返回，并返回 OK；否则返回 ERROR
     if( Q.rear == Q.front )   return ERROR;   // 空队列，无数据元素可出队
     e = Q.pBase[Q.front];                     // 将队头元素赋值给 e
     Q.front = (Q.front+1)%MAXQSIZE;       // 队头指针指向新的队头位置
     return OK;
} //DeQueue
```

循环队列巧妙地将一维数组首尾相连，其队头出队和队尾入队的操作不需要移动其他元素，因此时间复杂度都为$O(1)$。

3.8 队列的链式表示与实现

采用链式表示的队列称为**链式队列**（linked queue）（图 3.8）。与链式栈不同的是，链式队列的入队和出队在单链表的两端，需要增加一个指向单链表表尾的指针，显然，从表尾插入新结点并从表头删除结点较为方便。这样，入队和出队的时间复杂度也为$O(1)$。

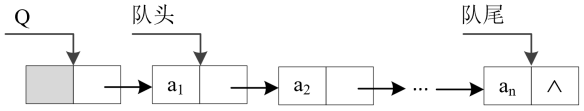


图 3.8 链式队列结构示意图

链式队列结点的存储结构与单链表结点一致，定义如下：

```
typedef struct QNode
{
   ElemType      data;          // 数据域
   struct QNode   *next;        // 指针域
} QNode, *QueuePtr;
typedef struct LinkQueue
{
   QueuePtr  front;             // 队头指针
   QueuePtr  rear;              // 队尾指针
} LinkQueue;
```

链式队列的大部分操作与单链表操作类似，这里只给出初始化队列、入队和出队的操作方法。

1. 初始化操作

算法 3.18 链式队列的初始化操作

```
Status InitQueue( LinkQueue &Q )
```

```
{    // 构造一个空的链式队列 Q
     // 1. 申请头结点
     Q.front = Q.rear = (QNode *) malloc( sizeof(QNode) );
     if ( Q.front == NULL ) exit(OVERFLOW); // 内存申请失败
     Q.front->next  = NULL;
     return OK;
} //InitQueue
```

2. 加工型操作：入队和出队

算法 3.19 链式队列的入队操作

```
Status EnQueue( LinkQueue &Q, ElemType e )
{    // 将数据元素 e 插入队列成为新的队尾
     // 1. 申请新的结点 s
     s = (QNode *) malloc( sizeof(QNode) );
     if ( s == NULL ) exit(OVERFLOW); // 内存申请失败
     s->data   = e;
     s->next   = NULL;
     // 2. 将结点 s 链接到队尾结点之后
     Q.rear->next = s;
     Q.rear = s;                // 更新队尾指针指向新的队尾
     return OK;
} //EnQueue
```

算法 3.20 链式队列的出队操作

```
Status DeQueue( LinkQueue &Q, ElemType &e )
{    //将队头元素数据元素出队，并用 e 返回，若是空队列，返回 ERROR
     if ( Q.front == Q.rear ) return ERROR;  // 空队列
     //1. 将队头数据元素赋值给 e
      p = Q.front->next;
      e = p->data;
     //2. 删除队头结点
     Q.front->next  = p->next;
     if ( Q.rear == p ) Q.rear = Q.front;       // 如果已是队尾，则要更新队尾指针
     free(p);
    return OK;
} //DeQueue
```

在一些特定场合下，还有一些扩展的队列表示，例如双端队列、优先级队列等。有兴趣的读者可以参阅相关文献。

3.9 队列的应用举例

在日常生活中，队列的应用随处可见：在银行、饭店、理发店、车站等都需要排队。令人印象深刻的还有节假日在网上抢购火车票的场景。平时，由于网上排队买票的人数在正常的合理范围内，服务器的反应较为迅速；但一到传统节假日，由于抢票人数众多，服务器的处理能力难以追上排队人数的增加速度，这将导致网站严重拥塞。可见，如果队列的出队速度与入队速度不能很好地匹配，就会影响用户体验。在平时涉及队列的应用设计中，这是值得考虑的因素。

这里列举一个工程应用中常遇到的问题。

例 3.6 温度值计算 一个温度传感器每秒钟可报 10 次温度值。应用系统的要求是每秒读取一次温度值，同时为了消除传感器误差，要求在读取温度值时将温度传感器当前时刻前 100 次的值取平均值输出。

显然，需要在应用系统中维护一个历史数据队列，保存当前时刻前 100 次的温度值，然后计算平均值并输出。相应操作见算法 3.21 和算法 3.22。

算法 3.21 传感器温度值加入队列

```
gSum = 0; // 队列元素求和值
Status AddTempToQueue( SeqQueue &Q, ElemType e )
{    // 传感器温度值加入队列
     if( (Q.rear+1)%MAXQSIZE == Q.front )        // 若队列满，队头先出队
     {
          gSum -=  Q.pBase[Q.front];                  // 将队头最久历史值从 gSum 中减除
          Q.front = (Q.front+1)%MAXQSIZE;
     }
     Q.pBase[Q.rear] =  e;                       // 插入元素 e 作为新的队尾元素
     gSum  += e;                                 // 将元素 e 累加到 gSum 中
     Q.rear = (Q.rear+1)%MAXQSIZE;       // 队尾指针指向队尾元素的下一个位置
     return OK;
} //AddTempToQueue
```

算法 3.22 从队列中输出符合要求的温度值

```
Status OutputTemperature( SeqQueue &Q, ElemType &val )
{    // 从队列中输出符合要求的温度值
     if ( (rear+MAXQSIZE-front)%MAXQSIZE < HIST_TIMES ) return DATA_NOT_ENOUGH;  // 队列中历史数据未到 100 次
     val  = gSum / HIST_TIMES;
     return OK;
} //OutputTemperature
```

3.10 小结

本章讨论了栈和队列，包括其抽象数据类型的描述、顺序表示与实现和链式表示与实现，以及相关的应用举例。

栈限定于在线性表的一端进行压栈和弹栈操作。采用顺序表示实现时，可将栈顶设置在顺序表的表尾；采用链式表示时，可将栈顶设置在单链表的表头。这样恰好分别满足顺序表和单链表插入和删除操作的最好情况。栈结构的最大特性是“后进先出”，因此它可以用于逆序输出、最近匹配与比较、递归与回溯等。栈在计算机系统中的应用最为广泛。

队列只能在线性表的一端入队、另一端出队。在采用顺序表示和链式表示时都无法直接使用线性表来达到最好的情况，需要进行适当改造：在采用顺序表示时，将原来的顺序表变成循环形式以满足入队和出队操作的需要；在采用链式表示时，增加一个指向单链表表尾的指针，将队列的入队操作放在单链表的表尾、队列的出队操作放在单链表的表头。当然，由于增加了表尾指针，在进行入队操作时，每次都需要修改表尾指针使其指向新的队尾；在进行出队操作时，要判断是否为最后一个元素出队，是否需要因此而修改表尾指针。可见，在增加一个辅助指针以带来便利的同时，也要防范引入后的“副作用”。队列结构的特性是“先入先出”，这在生活中较为普遍。特别是在分析最近的历史数据时，队列的最新数据入队和最旧数据出队恰好能满足其约束。

3.11 练习

1. 设一个栈的输入序列为 A、B、C、D，则借助一个栈所得到的输出序列不可能是（）

A. ABCD　　B. DCBA
C. ACDB　　D. DABC

2. 判定一个顺序栈 st 为（元素个数最多为 MaxSize）空的条件为（）
A. st.top==st.base　　B. st.top != st.base
C. st.top != MaxSize　　D. st.top == st.base+MaxSize

3. 用一个一维数组 S（大小为 MaxSize）作为两个栈的共享空间。请问如何设置？
A. 两个栈的栈底在数组中间，向两边进行扩展.
B. 两个栈的栈底分别设置在数组的两端，栈顶向数组中间移动。

4. 栈的插入和删除操作在（）进行
A. 栈顶　　B. 栈底
C. 任意位置　　D. 指定位置

5. 已知一个栈的进栈序列为 p1,p2,p3,…,pn，其输出序列为 1,2,3，…n. 若 p3 = 1，则 p1 的值（）
A. 一定是 2　　B. 可能是 2
C. 不可能是 2　　D. 一定是 3

6. 在一个具有 n 个单元的顺序栈中，假定以地址低端（即 0 单元）作为栈底，以 top 作为栈顶指针，当做出栈处理时，top 变化为( )
A. top 不变　　B. top = 0
C. top--　　D. top++

7. 栈和队列的共同特点是( )
A. 只允许在端点处插入和删除元素　　B. 都是先进后出
C. 都是先进先出　　D. 没有共同点

8. 向一个栈顶指针为 hs 的带头结点的链栈中压入一个 s 结点时，应执行( )
A. hs->next = s;　　B. s->next = hs; hs = s;
C. s->next = hs->next; hs->next = s;　　D. s->next = hs; hs = hs->next;

9．设计一个判别表达式中左、右括号是否配对出现的算法，采用（ ）数据结构最佳。
A. 线性表的顺序存储结构　　B. 队列
C. 线性表的链式存储结构　　D. 栈

10. 设栈 S 和队列 Q 的初始状态为空，元素 e1, e2, e3, e4, e5, e6 次通过栈 S，一个元素出栈后即进入队列 Q，若这 6 个元素出队的序列是 e2, e4, e3, e6, e5, e1, 则栈 S 的容量至少应该是（）
A. 6　　B. 4
C. 3　　D. 2

11. 对于链式队列，在执行入队操作时，（ ）
A. 仅修改头指针　　B. 仅修改尾指针
C. 头、尾指针都要修改　　D. 头、尾指针可能都要修改

12. 为解决计算机主机与打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区。主机将要输出的数据依次写入缓冲区，而打印机依次从缓冲区中取出数据。该缓冲区的逻辑结构应该是（）

A. 栈　　B. 队列

C. 树　　D. 图

13. 循环队列存储在数组 A[0…n]，则入队时的操作为（）

A. rear=rear+1　　B. rear=(rear+1)mod(n-1)

C. rear=(rear+1)mod n　　D. rear=(rear+1)mod(n+1)

14. 若用数组 A[0..5]来实现循环队列，且当前 rear 和 front 的值分别为 1 和 5，当从队列中删除一个元素，再加入两个元素后，rear 和 front 的值分别为（）

A. 3 和 4　　B. 3 和 0

C. 5 和 0　　D. 5 和 1

15. 设一个循环队列 Q[maxSize]的队头指针为 front, 队尾指针为 rear, 队列最大容量为 maxSize, 该队列的队满条件是（ ）

A. Q.front == Q.rear;　　B. Q.front+Q.rear>=maxSize

C. Q.front == (Q.rear+1)%maxSize;　　D. Q.rear == (Q.front+1)%maxSize;

16. 设一个循环队列 Q[maxSize]的队头指针为 front, 队尾指针为 rear, 队列最大容量为 maxSize, 该队列的队长计算为（ ） 。

A. Q.rear-Q.front　　B. Q.rear-Q.front+1

C. (Q.rear-Q.front) % maxSize + 1　　D. (Q.rear-Q.front+maxSize) % maxSize

17. 队列的插入操作是在队列的_____进行，删除操作是在队列的_____进行。

18. 分别用顺序存储、链式存储结构实现栈的基本操作，并验证测试。

19. 分别用顺序存储、链式存储结构实现队列的基本操作，并验证测试。

20. 采用非递归方法实现汉诺塔问题求解的算法，并输出每步盘子移动的情况。

21. 利用队列实现杨辉三角的打印，给出相应的实现算法。

22. 利用队列进行银行营业窗口的模拟。例如，通过预设每个时间段的客流量、每个客户处理时间的随机模拟，评估各时间段开设多少个窗口可以满足正常服务的需要。

23. 迷宫问题

[问题描述]

利用栈操作实现迷宫问题求解。

[基本要求]

（1）从文件中读取数据，生成模拟迷宫地图，不少于 20 行 20 列。

（2）给出任意入口和出口，显示输出迷宫路线。

3.12 延伸阅读

栈作为数据结构最早进入计算机科学，应归功于计算学科的先驱 Alan M. Turing。1974 年 Donald E. Knuth （图灵奖得主，杰出的计算机科学家）在其著作中指出[1]，Turing 早在 1946 年的工作为子程序的链接问题提出和发展了栈技术[2]，当初采用术语“bury”和“unbury”描述对子程序的调用和返回。

队列作为数据结构何时出现几无可考，但应与排队论密切相关。据文献[3]所考，现代排队论由 20 世纪初丹麦数学家和工程师 Agner K. Erlang 首创[4, 5]，并经 Pollaczek 和 Crommelin[6-9]发展，深入研究了队列的形成、运行、提供服务方式等，形成了系统的理论。20 世纪 40 年代 John von Neumann 研发的 EDVAC 计算机中“存储程序”的流程控制应有队列的雏形[10]；20 世纪 60 年代多道操作系统出现后，任务与进程调度策略采用队列来实现先来先服务（FCFS）、Round Robin（RR）等调度算法和其他的资源管理。

参考文献：

[1] KNUTH D E. The art of computer programming[M]. Pearson Education, 1997.
[2] TURING A M. Proposals for development in the mathematics division of an automatic computing engine (ACE) [R], 1946.
[3] JANSSEN A J E M, LEEUWAARDEN J S H V, Back to the roots of the M/D/s queue and the works of Erlang, Crommelin and Pollaczek[J], Statistica Neerlandica, 2008, 62(3): 299-313.
[4] ERLANG A K. The theory of probabilities and telephone conversations[J]. Nyt Tidsskrift for Matematik B, 1909, 20: 33-39.
[5] ERLANG A K. Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges[J], Post Office Electrical Engineer’s Journal, 1917, 10: 189-197.
[6] POLLACZEK F. Über eine aufgabe der wahrscheinlichkeitstheorie I [J]. Mathematische Zeitschrif. 1930, 32: 64-100.
[7] POLLACZEK F. Über eine aufgabe der wahrscheinlichkeitstheorie II [J]. Mathematische Zeitschrif. 1930, 32: 729-750.
[8] CROMMELIN C D. Delay probability formulas when the holding times are constant[J]. Post Office Electrical Engineers’ Journal, 1932, 25: 41-50.
[9] CROMMELIN C D. Delay probability formulae [J]. Post Office Electrical Engineers’ Journal, 1934, 26: 266-274.
[10] NEUMANN J. The first draft of a report on the EDVAC [R]. 1945.